AI Agent 应用精细化评测：评测体系设计与工程实践

阿里技术 2026年9月2日 03:11 浙江 46人

以下文章来源于AliExpress技术 ，作者砚东

AliExpress技术
阿里国际速卖通技术部（AliExpress Tech）官方技术号，呈现速卖通技术部在国际...



> *Good evaluations help teams ship AI agents more confidently. Without them, it's easy to get stuck in reactive loops—catching issues only in production, where fixing one failure creates others.*
> ——— *《Demystifying evals for AI agents》*

这是2026年的第 57 篇文章

（ 本文阅读时间：约 20 分钟 ）

01
引言：Agent 需要评测什么？

1.1 从“能用”到“好用”的距离

近年来，大模型驱动的 AI Agent 在各行各业加速落地。然而，当一个 Agent 应用上线后，团队往往面临一个尴尬的局面：用户反馈“回答不对”、“反应太慢”、“总是答非所问”，但开发者却很难定位问题的根因。

是意图理解出了偏差？是路由决策选错了路径？还是知识检索漏掉了关键信息？抑或是工具调用传错了参数？当多个模块串行协作时，端到端的黑盒评测只能告诉你结果不对，

AI Agent评测体系

出了问题。

1.2 传统评测的局限性

回顾 NLP 领域的评测历史，BLEU、ROUGE 等指标曾是机器翻译和文本摘要的标准评测手段。但到了 Agent 时代，这些面向“文本相似度”的指标已经力不从心——Agent 并非“标准答案生成器”，而是一个“理解意图、做出决策、调用工具、完成任务”的多步骤自主决策系统。

团队之前的评测工作仅仅关注的是 Agent 输出的最终答案质量，通过预设几类“文本相似度”的指标利用 LLM 进行打分，可以简单验证 Agent 已初步具备较好的基础能力，但这种做法面临以下根本性问题：

- **评测粒度太粗：**综合评分掩盖了 Agent 工作链路的真实表现，当两个版本 Agent 的综合评分相近时，开发者无法判断差异来自“某环节显著提升、另一环节明显退化"还是“各环节表现均匀波动”。
- **无法检测幻觉：**文本相似度指标可能会给一个看似合理但实际编造的答案打高分，幻觉内容在语法、逻辑和用词上往往是完美的，LLM 只能看到“表述是否相似”，无法验证“内容是否准确”。
- **忽略中间过程：**只对比最终答案，等于把 Agent 当作黑盒。无法发现 Agent 是否走了弯路——比如是否做了冗余的工具调用、是否在某个关键步骤出现了偏差但最终“蒙对了”答案。
- **掩盖成本效率：**两个 Agent 可能输出同样质量的答案，但一个调用了3次工具、耗时2秒，另一个调用了15次工具、耗时15秒，单一的质量评分完全掩盖了这种资源消耗差异，但这对生产环境也同样重要。
- **缺乏多轮评估：**真实 Agent 往往是多轮对话的，评测却常常是单轮问答。无法衡量 Agent 是否会在多轮交互中丢失上下文、是否重复提问已确认的信息、是否在用户纠正后依然固执己见。
- **真实场景脱节：**评测集往往是人工构造的“干净”问题，而生产环境中的用户输入充满歧义、口语化表达和上下文依赖。在一类评测集上拿到高分的 Agent ，面对真实用户的模糊请求时可能频繁失败。

对之前的评测感兴趣可以继续阅读这篇实践工作：《全球化商品中心智能答疑Agent实践》

1.3 本文的解法

粗粒度的“通过/不通过”已无法支撑业务快速迭代的需要——我们想要知道它在哪个环节、因为什么问题、多大的概率出错。为此，在商品中心 Agent 应用架构升级的过程中，本文构建了一套AI Agent 应用精细化评测体系，其设计遵循三个核心原则：

- **评测面向架构：**商品中心Agent 由感知、规划、记忆、工具四大核心模块组成，评测也应按同样的结构逐层拆解，让每一层的质量都可独立观测。
- **指标面向行动**：每一项指标都是一份“诊断报告”而非“成绩单”。它的价值不在于给出多高的分数，而在于下降时能够精准告诉开发者：是该修改 Skills、替换 MCP 工具、还是优化知识检索。
- **能力面向产品**：评测不应是上线前跑一次就扔掉的一次性脚本，而应是像监控告警一样能够可持续运行、可横向对比、可追溯演进的基础设施。

AI Agent评测体系

本文将从指标体系设计、数据集工程、评测方法论、执行引擎到可视化评测平台，完整分享这套评测体系的设计思考与工程实践。

02
评测体系总览：从黑盒到白盒

2.1 认识 Agent 的内部结构

在设计评测体系之前，首先需要理解被评测对象的内部结构。商品中心的 Agent 应用由四个核心模块协作运行：

1. **感知模块：**负责接收用户输入，通过 LLM 判断问题是否可被已注册的 Skill 直接处理，实现对用户问题的语义理解和意图识别。同时提供降级策略，未命中时自动降级至知识库检索路径，提升用户的交互体验。
2. **规划模块：**负责基于感知结果制定执行策略，决定是否需要调用工具、是否检索知识库、如何存储记忆等，通过持久化配置定义执行逻辑，实现 Skill 场景与 RAG 场景的自动分流，提升系统可维护性与扩展性。
3. **记忆模块：**负责为 Agent 提供短期和长期的记忆能力，短期记忆基于会话 ID 维护多轮对话上下文，采用滑动窗口机制截取最近 N 轮历史，并注入用户画像信息；长期记忆则对接 RAG 知识库，将检索结果作为上下文注入 Prompt，增强回答的准确性和专业度。
4. **工具模块：**负责为 Agent 提供工具接入管理能力，一是通过 MCP 协议自动发现并注册远程工具，支持多 Server 配置与统一调用；二是基于文件系统的 Skill 管理，按 Agent 维度隔离，支持热更新下发。两类工具统一封装为标准回调接口，由 Agent 按需调度执行。



对 Agent 细节内容感兴趣可以继续阅读这篇实践工作：国际化IC-基于Spring [illegible] 能答疑实践

AI Agent评测体系

2.2 为什么要拆到模块级别去评测?

假设 Agent 任务完成率从 85% 掉到了 60%，原因可能是：

- 感知模块把“查 Trace”错误识别为知识库问答（意图识别错误）
- 规划模块在应该调用 Skills 时选择了 RAG 检索（路由决策错误）
- 记忆模块在多轮对话中丢失了上一轮提到的商品 ID（记忆丢失错误）
- 工具模块错误调用了 MCP 或工具参数填充错误（工具调用错误）

端到端评测本质上是一个黑盒测试，就像是把车开上测试跑道：它不关心发动机转速、变速箱换挡逻辑或刹车片磨损程度，只回答一个用户视角的问题——这辆车能不能从 A 地开到 B 地。模块级评测则是白盒诊断：把车架上升降台，看发动机、刹车、油路各自的状态，关心车辆能不能安全、准时、经济地抵达目的地。端到端评测定义了“好车”的标准，模块级评测提供了“把车修好”的路径，两者应该相辅相成。

2.3 为什么需要“质量 × 成本 × 性能”指标?

一个“回答正确但耗时 30 秒”的 Agent 不是好 Agent；一个“回答正确但每次消耗 10 万 Token”的 Agent 不可持续。Agent 最终要部署到生产环境服务真实用户，评测框架必须同时回答三个问题：答案好不好？成本高不高？速度快不快？

因此，本文将 AI Agent 应用评测设计为三个维度：

- **质量维度**：答案是否正确、完整、忠实于知识来源，是否出现幻觉。这是底线，但不是全部。
- **成本维度**：每次请求触发了几次模型调用、调用了几轮外部工具、消耗了多少 Token。
- **性能维度**：从用户按下回车到看到第一个字的时延，以及到完整回答结束的时延，直接决定了用户体验。

这三个维度理论上缺一不可，只追求质量，可能在成本和延迟上失控；只优化速度，可能在复杂问题上过于简化；只看成本，可能导致准确率断崖式下跌。健康的 Agent 应是三个维度在当前业务场景下的最优平衡。

2.4 评测范围、数据集与指标的映射

参考已有评测工作，本文将整体评测分为两个部分：

端到端评测：主要关注 Agent 在接收到单一用户问题后，能否自主、有效地完成整个任务流程，并达到预期的输出效果，从任务完成质量、任务成本消耗、任务完成效率和安全鲁棒性四个维度综合衡量 Agent 的整体表现。

核心模块评测：主要关注核心模块的评测（感知、规划、记忆、工具），用于深入分析Agent的内部工作机制和潜在瓶颈。每个模块从能力、效率和有效性等多个评测维度进行细粒度评估，帮助定位具体问题根因。

AI Agent评测体系



不同的评测目标，需要不同的数据集和指标组合，而指标本身的数据来源也不尽相同。像首响延迟、Token 消耗量、模型调用次数这类成本与性能指标，可以通过系统埋点直接采集；而任务完成率、意图识别准确率、工具调用成功率、忠实性等质量类指标，则需要依赖评测集来判定。

本文通过预先定义评测范围、数据集与指标的映射开展评测工作。根据评测范围自动装配对应的评测数据集，同时拉取该范围相关的埋点指标。评测范围一变，数据集和埋点指标自动跟着调整，无需人工逐个对接数据源。通过“选择评测范围 → 自动装配数据集 → 自动筛选对应指标”，可以使评测过程简单而灵活。

评测模式	覆盖的评测数据集	覆盖的埋点指标
端到端评测	基础技能评测集、知识问答评测集、多轮对话评测集、异常输入评测集	模型调用次数、工具调用次数、Token消耗量、首Token延迟、端到端响应延迟、模块级延迟
核心模块评测	基础技能评测集、知识问答评测集、多轮对话评测集、工具调用评测集、多意图评测集、模糊意图评测集、长对话衰减评测集	模型调用次数、工具调用次数、Token消耗量、模块级延迟
感知模块评测	基础技能评测集、知识问答评测集、多意图评测集、模糊意图评测集	意图识别延迟
规划模块评测	基础技能评测集、知识问答评测集、工具调用评测集	规划决策延迟
记忆模块评测	知识问答评测集、多轮对话评测集、长对话衰减评测集	短期记忆注入延迟、长期记忆检索延迟
工具模块评测	工具调用评测集	工具调用延迟、热更新生效延迟

评测数据集	覆盖的评测指标

AI Agent评测体系

基础技能评测集	任务完成率、指令遵循能力、意图识别准确率/召回率/精确率、路由决策准确率、规划路径评分
知识问答评测集	任务完成率、幻觉率（忠实性）、意图识别准确率/召回率/精确率、降级触发准确率、路由决策准确率、规划路径评分、知识库检索决策准确率、长期记忆检索精确率/召回率
多轮对话评测集	多轮对话完成率、短期记忆保留率
异常输入评测集	异常输入处理率
工具调用评测集	工具调用决策准确率、工具调用准确率、工具调用成功率、参数映射准确率
多意图评测集	多意图识别率
模糊意图评测集	模糊意图澄清率
长对话衰减评测集	记忆衰减曲线

03
评测指标：核心指标的设计逻辑

3.1 端到端：六项“用户感知”指标

这是用户最直接能感受到的指标：

任务完成率是整个评测体系的北极星指标。它回答一个最基本的问题：Agent 是否成功完成了用户请求的任务？判断标准是语义层面的完成度——不要求字面匹配，但核心信息不能缺失。例如，用户问“商品为什么不可售”，Agent 只要正确输出了不可售原因的诊断分析即为通过，不要求措辞与标准答案完全一致。

多轮对话完成率评估 Agent 在多轮交互场景中的表现。Agent 是否能在 2–5 轮的对话中持续理解用户意图、维持上下文连贯并最终完成任务？这项指标需要对整组对话做整体评判，而非逐轮独立打分。

指令遵循能力关注的是形式正确性。某些 Skill 场景对回复格式有明确约束——需要分模块输出、需要包含特定的分析维度、需要表格化呈现。该指标用于衡量 Agent 是否按照业务约束的格式进行回答。

幻觉率 **（忠实性）**用于判断 Agent 是否忠实于其获取的知识库片段、工具返回的原始数据以及多轮对话中的历史上下文，检查是否存在“无中生有”或“擅自篡改”的行为。

> 在设计这项指标时明确划定了评测边界：只判断 Agent 有没有“忠实转述”，不判断 Agent 的信息来源本身对不对。原因在于，Agent 的本质是信息处理器，它通过检索知识库、调用工具获取信息，再基于这些信息进行推理和回答。如果知识库原文本身存在错误，应由知识库自身的评测体系来保障；只要 Agent 如实反映了它所看到的内容，即使最终答案与客观事实不符，也应判定为忠实。

AI Agent评测体系

> 具体判定标准上：对检索内容的合理归纳总结、基于上下文的解释性补充，均视为忠实；但凭空捏造具体的接口名、函数名、配置项等未在上下文中出现的实体，或给出与检索结果直接矛盾的核心信息，则判定为幻觉。

异常输入处理率评估 Agent 的安全鲁棒性。面对空输入、超长文本、乱码、特殊字符注入甚至 Prompt 注入攻击，Agent 是否能优雅降级——返回合理的引导提示而非崩溃或乱答。

用户满意度依赖线上反馈收集对 Agent 回复质量的评分，用于校准评测的盲区，为优化动作指明方向。

3.2 感知模块：六项“看懂没”指标

Agent 的第一步应该是“看懂用户在说什么”。感知模块的评测主要围绕**意图识别能力**和**降级触发能力**展开。

意图识别三件套借鉴了经典信息检索的评估范式。准确率衡量“识别出的意图对不对”，召回率衡量“该识别的都识别了吗”，精确率衡量“识别为 Skill 的请求真的可以由 Skill 处理吗”。三个角度交叉验证更能全面反映意图识别能力。

多意图识别率针对的是复合问题场景。当用户在一句话中混合了多个意图——比如“帮我查一下这个商品为什么不可售，顺便分析一下错误码 F_IC_SERVICE_QUERY_020 是什么意思”，Agent 能否完整拆解出所有意图?

模糊意图澄清率关注 Agent 面对模糊问题时的行为是否合理。当用户说“帮我看一下这个商品怎么回事”这类缺乏明确指令的问题时，合理的行为是主动追问以澄清意图，或基于合理推断给出有价值的回答；不合理的行为是答非所问或产生幻觉。

降级触发准确率衡量当用户的问题超出了所有 Skill 的处理范围时，Agent 是否正确触发了知识库检索兜底，而非强行路由到一个不匹配的 Skill?

3.3 规划模块：四项“想对没”指标

听懂意图之后，Agent 需要做出执行决策。规划模块的评测主要围绕**规划决策能力**和**规划决策合理性**展开。

路由决策准确率是规划模块的核心指标。对于每个用户请求，Agent 需要在“调用 Skill”和“检索知识库”之间做出正确选择。错误的路由决策会导致后续执行环节偏离正轨。

工具调用决策准确率关注当决定调用工具时，Agent 是否选对了工具种类？是该调 Trace 分析工具还是错误码查询工具?

知识库检索决策准确率关注当问题不属于任何 Skill 的能力范围时，Agent 是否正确触发了知识库检索?

规划路径评分通过人工评估 Agent 选择的执行路径是否最优，需要领域专家介入评判，验证整体策略的合理性。

3.4 记忆模块：四项“记住没”指标

Agent 不是无状态的函数调用，记忆能力直接影响多轮对话的用户体验。记忆模块的评测主要围绕**记忆注入能力**和**记忆注入有效性**展开。

短期记忆保留率评估多轮对话中的上下文连贯性。当用户在第三轮说“这个商品”时，Agent 是否还记得第一轮提到的商品 ID？本文在设计判断标准时特别强调了语义连贯性而非字面匹配——Agent 无需逐字复述历史实体名，只要回复在语义上延续了前文的讨论主题和上下文约束即可。

长期记忆检索精确率衡量 RAG 检索返回的知识片段的质量。检索回来的内容中，有多少与用户问题真正相关？本文对“相关”的定义是宽容的：直接回答问题的核心内容当然相关，提供理解问题所需的背景上下文（如相关概念定义、表结构说明）同样视为相关；只有与问题的业务领域完全无关的片段才判定为不相关。

长期记忆检索召回率关注知识库中应该被检索到的知识点，在 Agent 的最终回复中覆盖了多少？这项指标需要预先标注“应检索到的知识点列表”，然后判断 Agent 回复是否覆盖了这些知识点。

记忆衰减曲线能直观揭示 Agent 的记忆“保质期”。在不同轮次的长对话中，分别在第 2、3、5 轮插入需要引用历史信息的问题，统计各轮的记忆保留率。

3.5 工具模块：五项“做对没”指标

工具调用是 Agent 与外部世界交互的通道，工具模块的评测主要围绕**工具加载能力**和**工具调用能力**展开。

MCP & Skills 加载成功率是工具模块评测的底线指标，关注 Agent 启动或运行过程中，外部服务能否正确初始化并接入系统，是衡量工具调用的“前置条件”。

工具调用准确率检查 Agent 是否调对了工具。有没有多调或重复调用？有没有遗漏需要的工具？

工具调用成功率关注执行结果，检查工具调用是否成功返回了有效结果？

参数映射准确率用于衡量 Agent 是否正确地将用户意图映射为工具调用参数？比如用户说“查一下商品 1005007651467330 在韩国的可售状态”，Agent 传给工具的商品 id 参数是否正确？

3.6 成本与性能指标

成本指标包括四项：平均模型调用次数、平均工具调用次数、平均输入 Token 数和平均输出 Token 数。这些指标均通过 Agent 运行时埋点自动采集，按请求聚合统计。

性能指标覆盖两个层面：

AI Agent评测体系

- 端到端层面，主要关注**首 Token 延迟**和 **端到端响应延迟**。首 Token 延迟直接决定用户的“等待焦虑感”，即使完整回答还需要更长时间，快速出现的第一个字就能有效提升体验。
- 核心模块层面，主要关注**意图识别延迟**、**规划决策延迟**、**短期记忆注入延迟**、**长期记忆检索延迟**、**工具调用延迟**、**热更新生效延迟**。这些模块级延迟数据可以精确定位 Agent 性能瓶颈。

04
评测数据集：覆盖真实业务场景

4.1 数据集概览

本文结合真实业务场景构造了 8 类评测集，形成“基础覆盖 + 专项探测”的分层结构：基础技能与知识问答评测集验证 Agent 的核心能力闭环，是评测体系的基座；多轮对话、异常输入、工具调用、多意图、模糊意图和长对话衰减等专项评测集则从不同维度切入，定位感知、规划、记忆、工具等模块的具体短板。

评测集类型	构建方式	覆盖的评测指标	规模建议
基础技能评测集	按 Skill 维度构造，部分实时性分析场景通过 Mock 工具返回值来隔离外部依赖	任务完成率、指令遵循能力、意图识别准确率/召回率/精确率、路由决策准确率、规划路径评分	≥ 50 条
知识问答评测集	从知识库中抽取问答对，覆盖有明确答案、无答案需降级、部分匹配等场景	任务完成率、幻觉率（忠实性）、意图识别准确率/召回率/精确率、降级触发准确率、路由决策准确率、规划路径评分、知识库检索决策准确率、长期记忆检索精确率/召回率	≥ 100 条
多轮对话评测集	构造 2–5 轮的多轮对话链，覆盖上下文引用、话题切换、指代消解等场景	多轮对话完成率、短期记忆保留率	≥ 10 组
异常输入评测集	包含空输入、超长文本、乱码、特殊字符提问等	异常输入处理率	≥ 20 条
工具调用评测集	按工具维度构造，覆盖正确参数、缺失参数、错误参数等场景	工具调用决策准确率、工具调用准确率、工具调用成功率、参数映射准确率	≥ 20 条

AI Agent评测体系

多意图评测集	构造包含 2–3 个意图的复合场景问题	多意图识别率	≥ 20 条
模糊意图评测集	构造语义模糊、缺乏关键参数或指令不明确的问题	模糊意图澄清率	≥ 20 条
长对话衰减评测集	构造 5 轮以上长对话链，在关键轮次插入需引用历史信息的问题	记忆衰减曲线	≥ 10 组

4.2 评测用例构建规范

一条好的评测用例远不止“输入 + 期望输出”。本文为每条用例设计了丰富的标注字段，使一条数据可以同时供多个 Judge Task 使用，有效提高了数据的利用率。下面主要介绍下基础技能评测集、知识问答评测集、多轮对话评测集的构建规范和注意事项。

基础技能评测集按 Skill 维度构造，每条用例完整标注了评测链路所需的信息，不同的 Judge Task 各取所需。

```
{
  "id": "BF-TRACE-001",
  "sceneCode": "i18n-ic-trace-analyzer",          // 场景编码，用于分组统计
  "sceneName": "Trace排查",                       // 场景名称，可读性标识
  "userInput": "traceId: 2116440e17706430209582423d0733",
  "expectedSkill": "i18n-ic-trace-analyzer",  // 期望命中的 Skill → 意图准
  "expectedRoute": "SKILL_HIT",               // 期望路由方向 → 路由决策准
  "expectedIntent": "i18n-ic-trace-analyzer", // 期望意图类型 → 意图召回率
  "bizCode": "IC_PRODUCT",                    // 业务编码，构建 Agent 入参
  "datasetType": "BASIC_FUNCTION",            // 数据集类型，Judge 路由依据
  "evalMode": "E2E_MOCK",                     // 评测模式：E2E_MOCK / E2
  "mockDataId": "MOCK-TRACE-001",             // 关联的 Mock 数据 ID（没有
  "referenceOutput": "Agent应调用trace分析工具..." // 参考输出，模型评判依据
}
```

其中，evalMode 字段用于区分评测执行策略：

- E2E_MOCK（Mock 模式）：通过 mockDataId 关联预置的 Mock 数据，Agent 运行时注入 Mock 替代真实工具调用，确保评测结果可复现、不受商品状态的影响。适用于 Trace 排查、可售性分析、标签分析等依赖外部工具的实时性分析场景。
- E2E_REAL（真实模式）：不注入 Mock，Agent 直接调用真实服务。适用于不依赖实时性分析场景的 Skill 类评测（如错误码查询）和知识问答类评测，Mock 反而会失去评测意义。

由于 E2E_MOCK 模式下工具返回值是确定的，referenceOutput 只需描述 Agent 应有的行为和输出结构，而非具体的数据内容，以确保评测标准与 Mock 场景对齐。

AI Agent评测体系

> 之所以只描述行为而非具体内容，是因为基础技能评测集的评测目标是验证 Agent 的端到端行为链路（意图识别 → Skill 路由 → 工具调用 → 结果生成）是否正确，而非回答的具体措辞。Mock 数据已保证工具返回值确定，Agent 只要完成了正确的行为链路，基于确定的输入就能产出合理的输出。具体的输出质量（幻觉率、检索精确率/召回率）可以通过知识问答评测集单独验证。

Mock 数据按工具维度组织，每条 Mock 记录预设了各工具的返回结果。当评测执行到工具调用环节时，拦截器会根据当前工具名从 Mock 数据中查找对应的返回值，替代真实的远程调用。Agent 拿到的是与真实调用格式完全一致的结果，基于此继续推理和生成回答。这使得可以在不依赖任何外部环境的情况下，验证 Agent 从意图识别到最终回答的完整能力。

以 Trace 排查场景为例：

```
{
  "MOCK-TRACE-001": {
    "description": "sellerId为空导致空指针异常",
    "skillName": "i18n-ic-trace-analyzer",
    "mockResponse": {
      "execute_ae_script_582": "[{\"errorCode\":\"F_IC_SCENE_QUERY_015\
      "repo_vector_search": "[{\"content\":\"F_IC_SCENE_QUERY_015=Selle
    }
  }
}
```

知识问答评测集用于评测 Agent 基于知识库回答领域问题的能力，重点验证检索质量和回答的事实准确性。评测集全部采用 E2E_REAL 模式，不注入 Mock——因为知识问答的核心是验证 RAG 链路（检索 → 生成）的真实效果，Mock 会绕过检索环节，失去评测意义。

```
{
  "id": "KQA-BASE-001",
  "sceneCode": "default",                    // 场景编码，知识问答统一为 de
  "sceneName": "基础信息",                    // 场景名称，可读性标识
  "userInput": "商品和SKU有什么区别？请举例说明。",
  "referenceOutput": "商品（Product/Item）是指可以在平台上销售的实体或服务...'
  "expectedSkill": null,                     // 期望不命中任何 Skill
  "expectedRoute": "SKILL_MISS",             // 期望路由到知识库 → 路由决
  "expectedIntent": "knowledge_qa",          // 期望意图类型 → 意图准确率
  "expectedKnowledge": [                     // 期望检索到的知识点 → 长期
    "商品（Product/Item）是指可以在平台上销售的实体或服务，商品ID是商品的唯一标识
    "SKU（Stock Keeping Unit，库存量单位）是商品的最小销售单元，使用skuId进行表
  ],
  "bizCode": "IC_PRODUCT",
  "datasetType": "KNOWLEDGE_QA",
  "evalMode": "E2E_REAL"                     // 直接调用真实
```

AI Agent评测体系

```
}
```

另外，知识问答评测集中同时包含部分负例，用于验证 Agent 的边界能力——面对不该回答或无法回答的问题时，是否能正确识别并避免产生幻觉。例如在下面的用例中，deleteAllProducts 接口名看起来完全符合 IC 的命名风格，Agent 很容易基于已有的 saveProduct、queryProduct 等接口知识“推理”出一个看似合理但完全虚构的回答。这正是幻觉率（忠实性）检测要捕捉的问题。

```
{
  "id": "KQA-NEG-004",
  "sceneCode": "default",
  "sceneName": "负例-编造概念",
  "keywords": "不存在的接口,编造",
  "userInput": "IC的deleteAllProducts接口怎么调用？需要传哪些参数？",
  "referenceOutput": "IC中不存在deleteAllProducts接口，应明确告知未找到该接口
  "expectedSkill": null,// 不应命中任何 Skill
  "expectedRoute": "SKILL_MISS",// 应走知识库路由
  "bizCode": "IC_PRODUCT",
  "datasetType": "KNOWLEDGE_QA",
  "expectedIntent": "knowledge_qa",
  "evalMode": "E2E_REAL",
  "expectedKnowledge": []// 没有应检索到的知识点
}
```

多轮对话评测集衡量的是 Agent 在连续多轮交互中的上下文记忆能力——能否正确保持对话状态、记住前轮信息、并在跨场景切换时维持记忆连贯性。

该评测集的独有结构是一个有序的对话轮次数组（conversationChain）。评测执行器会依次向 Agent 发送每轮 userInput，保持同一会话上下文，模拟真实的多轮交互。每轮的 expectedContains 标注了该轮回复中应包含的关键词，供 Judge 逐轮检验 Agent 是否保持了正确的上下文信息。

```
{
  "id": "MT-001",
  "sceneCode": "canot_salable_reason",
  "sceneName": "多轮对话-可售性追问",
  "userInput": "商品1005007651467330在韩国不可售是什么原因",
  "referenceOutput": "该商品在韩国不可售，原因是...",
  "expectedSkill": "canot_salable_reason",
  "expectedRoute": "SKILL_HIT",
  "expectedIntent": "canot_salable_reason",
  "expectedContains": ["不可售", "sale_country_rule"],     // 整体层面的期
  "evalMode": "E2E_MOCK",
  "mockDataId": "MOCK-MT-001",
```

AI Agent评测体系

```
  "conversationChain": [                                    // 多轮对话链
    {
      "turnNumber": 1,
      "userInput": "商品1005007651467330在韩国不可售是什么原因",
      "referenceOutput": "该商品在韩国不可售，原因是sale_country_rule配置了黑
      "expectedContains": ["不可售", "sale_country_rule"]  // 每轮独立的期
    },
    {
      "turnNumber": 2,
      "userInput": "sale_country_rule和visible_country_rule有什么区别？",
      "referenceOutput": "sale_country_rule控制可售，visible_country_rul
      "expectedContains": ["可售", "可见", "黑名单", "白名单"]
    },
    {
      "turnNumber": 3,
      "userInput": "黑名单模式和白名单模式分别是怎么生效的？",
      "referenceOutput": "黑名单模式(B)：列表中的国家不可售...",
      "expectedContains": ["黑名单", "白名单"]
    }
  ]
}
```

4.3 基于 LLM 的评测集自动生成

手工编写数百条测试用例效率低且覆盖不足，因此，本文实现了从知识文档到评测用例的自动生成。

基于 Aone 文档知识工具集 能力，通过输入一篇语雀文档 URL 和目标数据集类型，系统会根据数据集类型特定的 Prompt 模板和具体文档内容，自动生成符合标注规范的结构化测试用例。例如，指定数据集类型为 KNOWLEDGE_QA（知识问答），生成器会自动产出包含 expectedKnowledge（应检索到的知识点）的用例。

当然，自动生成的用例质量参差不齐，需要经过人工审核后才能正式使用。“LLM 生成 → 人工审核 → 入库”的半自动化工作流，在保证质量的前提下将评测集构建效率提升了数倍。

05
Judge Task 设计：让大模型当可靠“裁判”

面对若干评测指标、数百条测试用例的评测规模，人工评测显然不可持续。传统的规则匹配（关键词、正则表达式）又无法处理语义等价的情况——“商品下架对应的事件标识”和“product-downshelf 消息类型”在语义上完全等价，但关键词匹配会判定为不一致。

为提升评测的可持续性和一致性，采用 LLM-as-Judge 作为核心自动化评测手段，其核心优势在于：它能在语义层面进行判断，理解“不同措辞表达同一含义”的等价关系。但

可靠的“裁判”，需要面向指标类型设计特定的评测任务（Judge Task）和提示词（Judge Prompt），从而输出与指标对应的可计算的结果。

5.1 Judge Task 的分层设计

本文将依赖评测集判定的指标评测任务归纳为六种类型，具体判断标准和量化方法如下：

【待插入图片】

5.2 Judge Prompt 的设计原则

好的 Judge Prompt 是评测质量的基础，结合实践经验，本文总结了四条设计原则：

第一，单一职责。 一个 Prompt 只评测一个指标。之前尝试过在一个 Prompt 中同时判断任务完成率和忠实性，结果两项指标的准确率都下降了。原因很简单：多任务混合会增加 LLM 的理解负担，导致判断质量下降。

第二，先推理后判断。 每个 Prompt 都要求 LLM 先输出 reasoning（思维链推理），再给出结论。这不仅提高了判断准确性，也让评测结果具备了可解释性——当 Judge 不通过时，能看到它的推理依据是什么。

第三，负例引导。 每个 Prompt 都包含典型的“通过”和“不通过”示例。例如任务完成率的 Prompt 中会给出这样的对比：用户问“商品 XXX 不可售原因”，Agent 输出了具体的不可售原因分析 → 通过；Agent 回答了商品基础信息但未分析不可售原因 → 不通过。负例引导能有效校准 Judge 的评判标准。

第四，结构化输出。 所有 Judge 的输出都要求严格的 JSON 格式，便于自动化解析和指标计算。在 Prompt 末尾明确给出输出格式模板，是确保 LLM 输出可解析的关键。

5.3 如何决定一个评测用例通过？

一种常见的做法是：把所有 Judge Task 的结果做 AND 运算——全部通过才算通过。这看似严谨，实则不合理。比如一个知识问答场景，Agent 准确回答了用户关于“商品不可售原因”的问题，内容完整、结论正确，但回复的格式没有严格按照参考输出的分段结构来组织——指令遵循判定为不通过。如果因为格式问题否定了一个内容正确的回答，评测结果的真实性将严重偏离。

本文的做法是为每种评测场景设定一个**主指标**，只看这一个指标来决定 case 是否通过。其他指标不影响最终通过判定，而是作为质量维度单独衡量，用于定位具体的能力短板。主指标的选取根据预先定义的评测范围分为评测数据集和评测模式，评测模式的优先级高于评测数据集。

评测模式	主指标	选择理由
端到端评测	任务完成率	端到端关注最终任务完成的效果
核心模块评测	任务完成率	关注模块协同后任务完成的效果
感知模块评测	意图识别准确率	感知的核心职责是

AI Agent评测体系

规划模块评测	路由决策准确率	规划的核心职责是决定走哪条路
记忆模块评测	短期记忆保留率	记忆的核心职责是记住对话内容
工具模块评测	工具调用准确率	工具的核心职责是准确调用工具

评测数据集	主指标	选择理由
基础技能评测集	任务完成率	最终没答好用户问题就是不通过
知识问答评测集	任务完成率	最终没答好用户问题就是不通过
多轮对话评测集	多轮对话完成率	需要评估整个对话流程，而非单轮表现
异常输入评测集	异常输入处理率	不存在“完成任务”，关键是合理应对
工具调用评测集	工具调用准确率	调对工具是前提，细节偏差可适当忽略
多意图评测集	多意图识别率	用户意图识别不全则后续执行必然缺失
模糊意图评测集	模糊意图澄清率	追问>猜测>幻觉，行为比对错更重要
长对话衰减评测集	记忆衰减曲线	专门考察记忆能力，以记忆保留为标准

5.4 一个容易被忽略的细节：路由错误时下游指标评测应当跳过

假设有一条知识问答类评测用例：“IC 商品下架后可售性字段会怎么变？”，标注的期望路由是 SKILL_MISS（走知识库检索）。但实际执行时，Agent 误将其路由到了某个 Skill——路由判断本身就错了。此时 Skill 直接生成了回答，RAG 检索根本没有被触发。于是触发以下指标评测时：

- 幻觉率（忠实性）：需要拿 Agent 回复与检索的知识库原文对照——但没有知识库原文
- 长期记忆检索精确率：需要统计返回的 chunk 中有多少相关——但没有返回任何 chunk
- 长期记忆检索召回率：需要检查期望 chunk 是否被覆盖——但覆盖的前提是检索执行了

如果仍然执行评测，这三项指标都会因为路由错误而被判定为不通过，而这三项指标衡量的是 Agent 回答生成和知识库检索的能力——它们本身并没有出错，只是根本没有被执行的机会。

本文的解决方案是：在每个依赖 RAG 数据的 Judge Task 执行前，先检测是否存在路由误触发（期望 SKILL_MISS 但实际 SKILL_HIT）。一旦检测到，相关的下游 Judge Task 自动标记为 error 并跳过统计，不计入分子也不计入分母。最终在统计通过率时，只有非 error 的结果才参与计算。

AI Agent评测体系

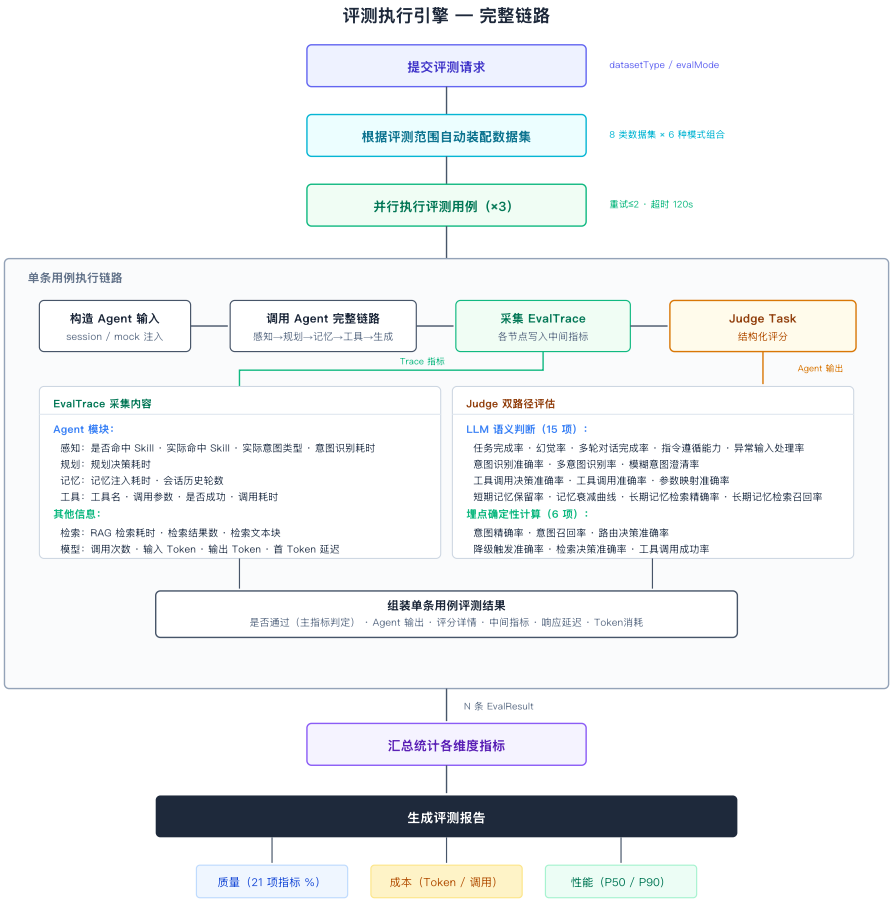
这样做的效果是：路由错误只会体现在路由决策准确率这一项指标上，而幻觉率、检索精确率、检索召回率只反映知识库检索和回答生成模块本身的能力，不会被上游的路由错误所污染。每项指标各司其职，一个模块的错误不会雪崩式地拖垮其他模块的数据。

06
评测执行引擎：从用例到报告

6.1 整体流程

评测执行的完整链路如下：

1. 提交评测请求：指定评测范围，包含 datasetType（评测数据集）和 evalMode（评测模式）。
2. 自动装配数据集：根据评测范围加载评测用例，每条用例包含用户输入、期望输出、Mock数据等。
3. 并发执行评测用例：单条超时120s，失败自动重试至多 2 次，兼顾执行效率与结果稳定性。
4. 单条用例执行链路：主要包含四个步骤：
   a. 构造输入：注入 sessionId、Mock数据，模拟真实用户会话环境；
   b. 调用 Agent 完整链路：感知 → 规划 → 记忆 → 工具 → 生成；
   c. 采集 EvalTrace：覆盖感知、规划、记忆、工具模块和 RAG 检索、模型消耗等维度；
   d. Judge Task 结构化评分：将 Agent 输出与 EvalTrace 提交给 Judge 并根据范围评估关联指标。
5. 生成评测报告：输出质量 × 成本 × 性能三大类指标，每条用例的通过与否由该数据集类型对应的主指标决定，确保评测结论聚焦核心能力。



6.2 数据集自动装配

评测的第一个问题是：给定一个评测范围，应该加载哪些评测用例？

本文设计了 15 种评测范围，覆盖从单个数据集到全模块评测的各个粒度。每种范围对应一组数据集的组合规则：

- 指定数据集（8 种）：直接加载对应的测试集，如 BASIC_FUNCTION 只加载基础技能评测集。
- 端到端评测（END_TO_END）：组合基础技能、知识问答、多轮对话、异常输入评测集，覆盖 Agent 的整体表现。
- 分模块评测（4 种）：例如感知模块评测加载基础技能、知识问答、多意图、模糊意图评测集，因为这 4 类数据集恰好覆盖了意图识别的各种边界场景。
- 核心模块评测（CORE_MODULE）：加载全部 8 类评测集，全面评测感知、规划、记忆、工具四个模块。

6.3 运行时指标采集

评测的一个核心挑战是：如何在 Agent 执行过程中采集中间指标，而不侵入 Agent 的业务逻辑？

本文设计了一个轻量级的 Trace 数据结构，随 Agent 的处理链路一起流转。在 Agent 执行过程中，各个节点在完成自己的工作后，将关键信息写入这个 Trace：

AI Agent评测体系

- 感知节点写入：是否命中 Skill、实际命中的 Skill 名称、实际意图类型、意图识别耗时
- 规划节点写入：规划决策耗时
- 记忆节点写入：记忆注入耗时、会话历史轮数
- 工具节点写入：工具调用记录（工具名、调用参数、是否成功、调用耗时）
- RAG节点写入：检索耗时、检索结果数量、检索到的原始文本块
- 其他信息统计：模型调用次数、输入/输出 Token 消耗等

Agent 执行完毕后，这个 Trace 和 Agent 的最终输出一起交给 Judge 引擎。Judge 不仅参考 Agent 的输出文本，还会利用 Trace 中的中间数据——例如“Agent 是否确实调用了某个工具”不需要从回复文本中推断，直接看 Trace 中的工具调用记录即可。

6.4 多轮对话评测的特殊处理

多轮对话是评测中最复杂的场景，需要处理几个特殊问题：

会话上下文共享：每组多轮对话生成独立的 session 标识符，各轮次共享同一个标识符，确保 Agent 能读到完整的对话历史。

轮次间同步：每一轮执行完毕后，需要等待会话持久化完成（通常 2 秒），再发起下一轮。否则下一轮可能读不到上一轮的对话历史，导致 Agent 并非不具备记忆能力，只是没读到该读的数据。

指标累计：多轮对话的模型调用次数、Token 消耗等成本指标需要逐轮累加，反映整组对话的总开销。

整体评判：所有轮次的输出拼接后统一交给 Judge 评估，而非逐轮独立评判。因为多轮对话的质量是一个整体，第一轮正确但第三轮失忆，整组对话应该判为失败。

6.5 两种评测模式

本文设计了两种互补的评测模式，解决评测中「真实性」与「可复现性」的矛盾：

端到端真实模式（E2E_REAL）：Agent 走完整的线上链路，包括真实的 LLM 调用、真实的工具调用和真实的知识库检索。这种模式能真实反映 Agent 在生产环境下的表现，但结果会受外部依赖抖动的影响：工具接口偶发超时、知识库刚好在更新、LLM输出的随机性——这些因素都可能导致同一条用例两次评测结果不同。

端到端 Mock 模式（E2E_MOCK）：注入预设的工具调用返回值，Agent 的推理和决策仍然是真实的，但工具调用的结果是固定的 Mock 数据。这种模式消除了外部依赖的不确定性，保证评测结果可复现。

两种模式各有适用场景：Mock 模式适用于 Trace 排查、可售性分析、标签分析等依赖外部工具的实时性分析场景；Real 模式适用于不依赖实时性分析场景的 Skill 类评测（如错误码查询）和知识问答类评测。

AI Agent评测体系

6.6 重试与容错

评测过程中不可避免地会遇到各种异常——LLM API 偶发超时、服务暂时不可用等，重试与容错策略为：

- **执行异常**（超时、网络错误等）：最多重试 2 次，重试间隔 3 秒，重试通常能恢复。
- **评估失败**（Judge 判定不通过）：不重试，因为这是 Agent 能力本身的问题，重试不会改变结果。
- **超时保护**：单条用例 120 秒超时，防止单个异常用例阻塞整批评测。

6.7 并行执行与异步管理

批量评测采用 3 线程并行执行，这个并发度是在评测效率和 LLM API 限流之间权衡的结果——过高的并发度会触发 API 限流反而更慢，过低又导致一批评测耗时过长。

评测任务采用异步提交–轮询模式：

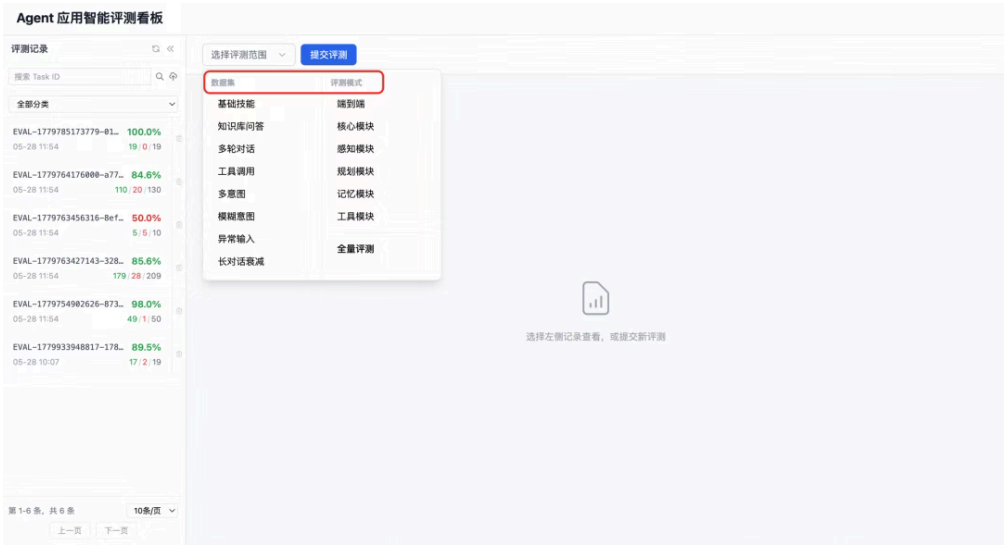
- 客户端提交评测请求，立即获得一个任务标识符（taskId）。
- 评测在后台异步执行，客户端通过轮询接口查询进度。
- 支持在运行中取消任务。

07
评测可视化：Agent 指标看板

本文初步构建了一个轻量级的评测看板，只需选择评测范围即可自动装配对应的评测数据集，同时拉取该范围相关的埋点指标。评测范围一变，数据集和埋点指标自动跟着调整，无需人工逐个对接数据源。通过“选择评测范围 → 自动装配数据集 → 自动筛选对应指标”，可以使评测过程简单而灵活。

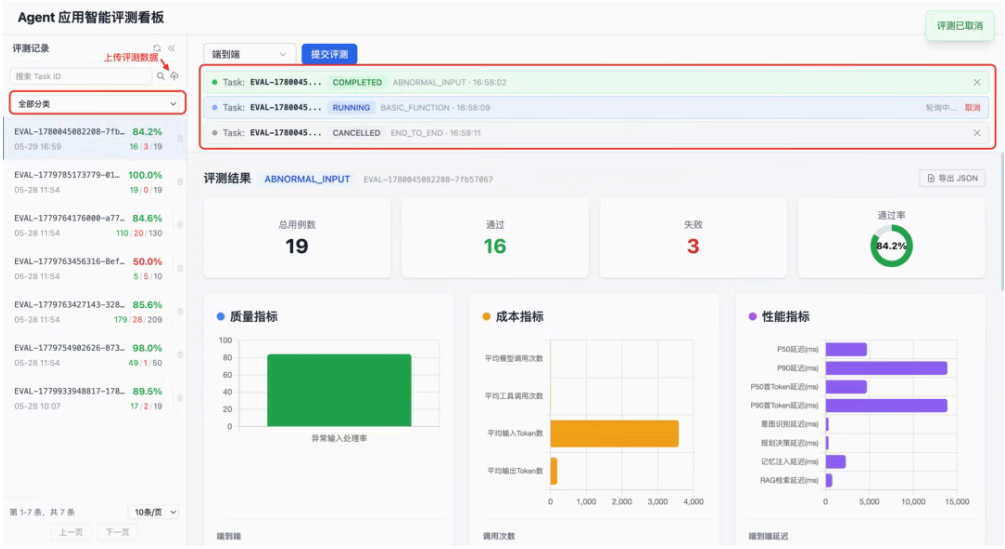
7.1 评测流程

评测范围：平台顶部提供评测范围选择器，分为“数据集”和“评测模式”两栏。数据集栏包含 8 种评测数据集（基础技能、知识库问答、多轮对话、工具调用、多意图、模糊意图、异常输入、长对话衰减），评测模式栏包含端到端、核心模块、感知模块、规划模块、记忆模块、工具模块和全量评测。



评测提交：选择后一键提交，平台实时显示运行状态（运行中/已完成/失败/已取消），支持取消运行中的任务。

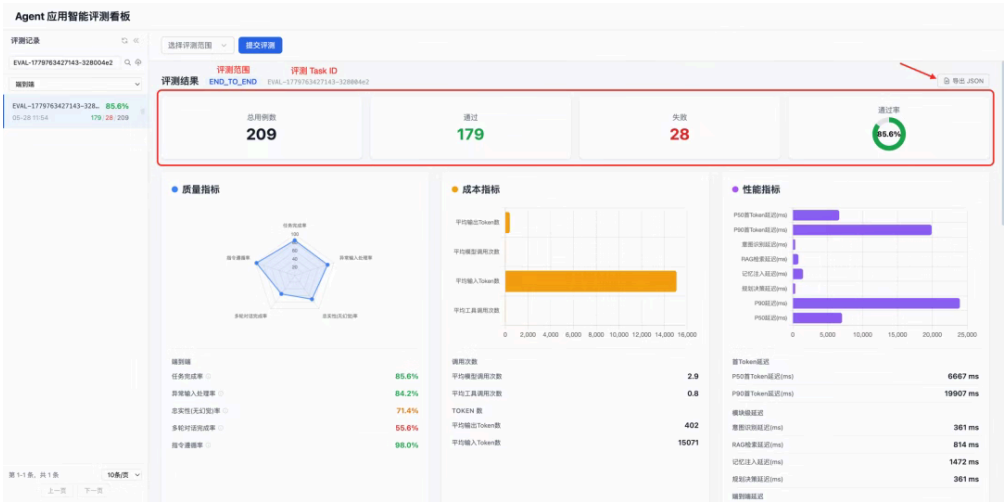
评测记录：左侧边栏展示所有历史评测记录，展示评测时间、通过率、通过/失败/总数等关键信息。支持按评测范围分类筛选、分页浏览。还支持 JSON 评测数据的导入，便于跨环境对比和离线分析。



7.2 多维指标

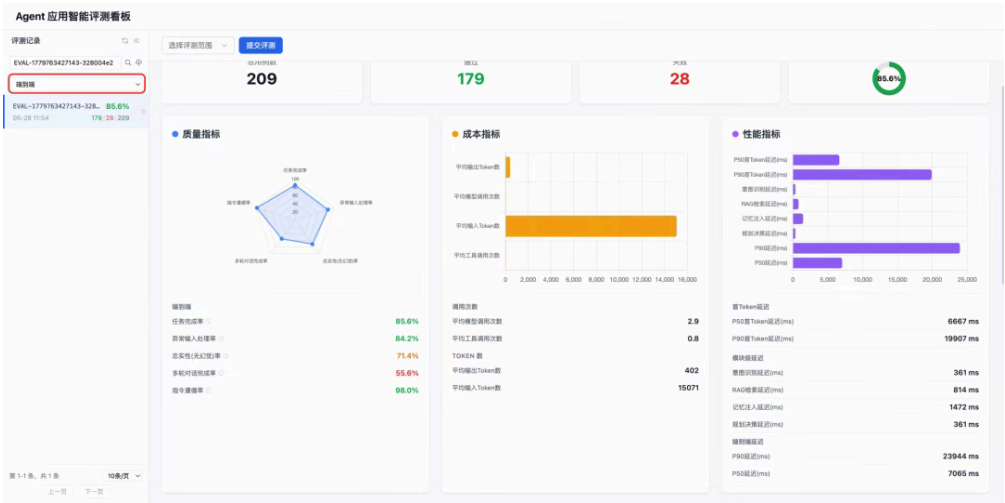
评测结果：评测完成后，下方首先展示总用例数、通过数、失败数和通过率，支持导出 JSON 评测数据。

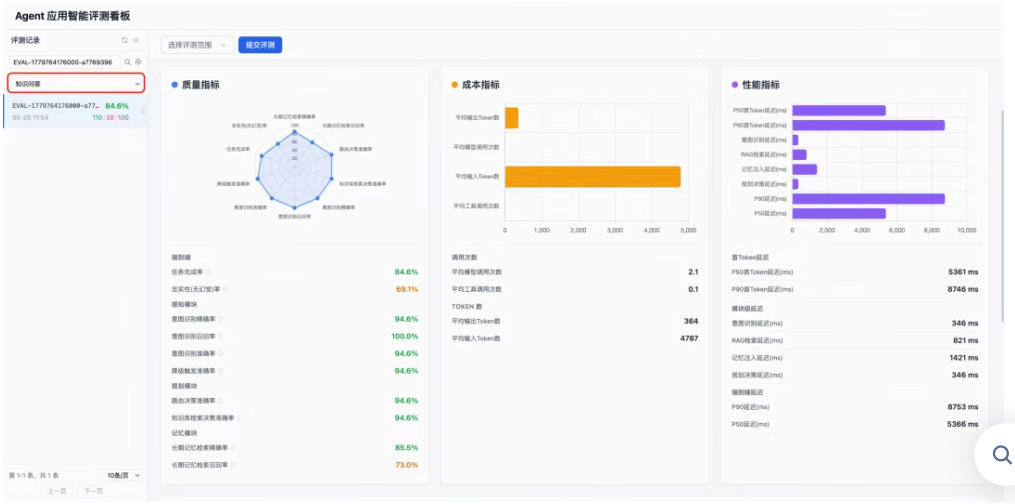
AI Agent评测体系



三维仪表盘：以三列布局展示“质量 × 成本 × 性能”评测指标，以端到端和知识库问答评测为例：

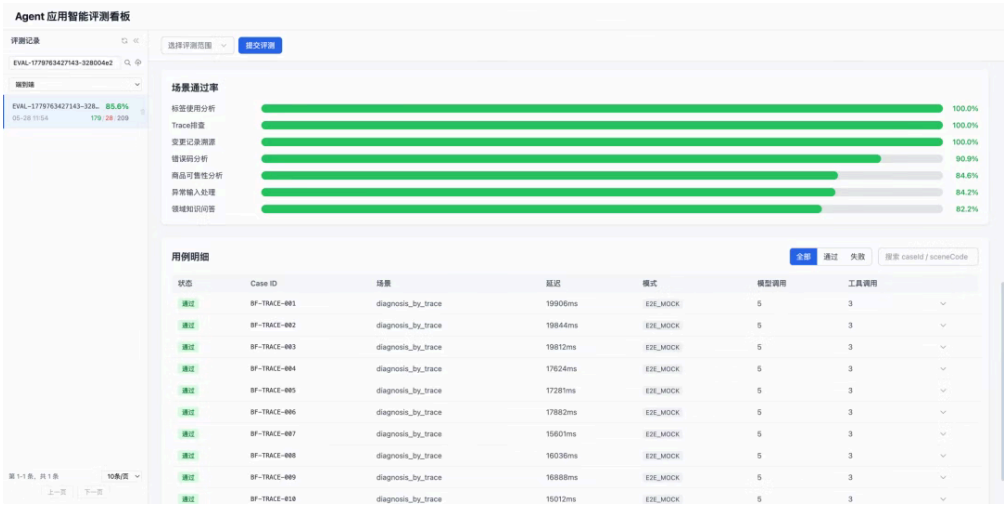
- **左栏质量指标**：以雷达图展示各质量指标的通过率，下方可按模块分组（端到端、感知模块、规划模块、记忆模块、工具模块）列出每项指标的具体数值，鼠标悬停可看到指标的详细描述。
- **中栏成本指标**：以横向柱状图展示调用次数和 Token 消耗，按“调用次数”和“Token 数”分类展示。
- **右栏性能指标**：以横向柱状图展示延迟数据，按“端到端延迟”、“首 Token 延迟”和“模块级延迟”分类展示。





AI Agent评测体系

场景通过率：以进度条的形式展示各业务场景（Trace 排查、错误码分析、标签使用分析、可见可售性分析、领域知识问答等）的通过率，按通过率从高到低排序，快速定位薄弱环节。

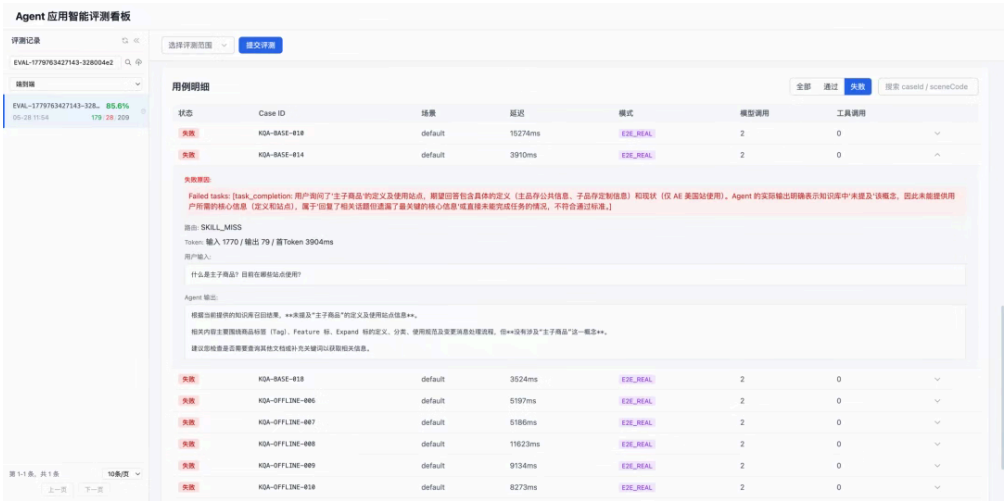


7.3 用例诊断

用例明细：详细的用例结果表格，支持通过/失败筛选、关键词搜索和分页浏览。每条用例可展开查看：失败原因、命中技能、路由结果、Token 消耗明细、用户原始输入和 Agent 完整输出，是定位具体问题的核心入口。

「失败用例一：知识问答」

Agent 的实际输出明确表示知识库中“未提及”概念，未能有效检索出用户所需的核心信息，未能完成任务。

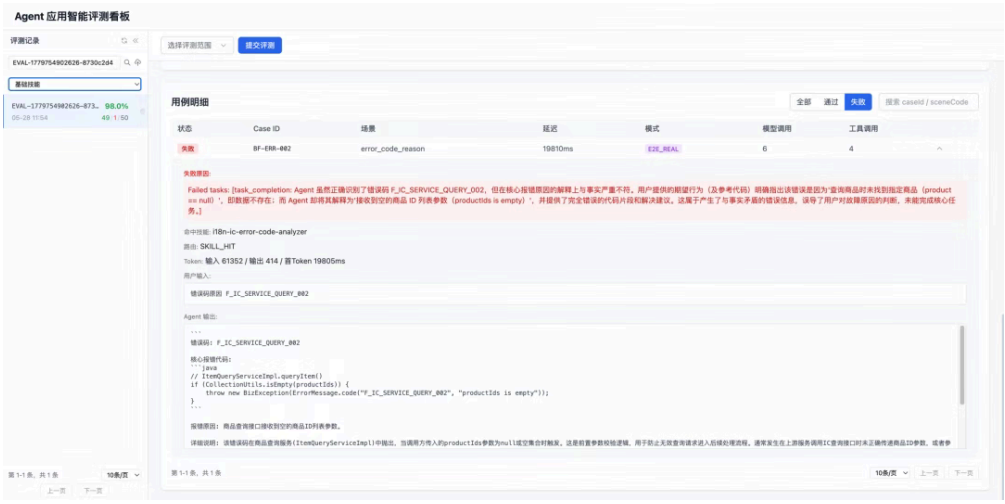


「失败用例二：错误码分析」

Agent 虽然正确识别了错误码 F_IC_SERVICE_QUERY_002，但在核心报错原因的解释上为“接收到空的商品 ID 列表参数（productIds is empty）”，并提供了完全错误的代码片段和解决建议。

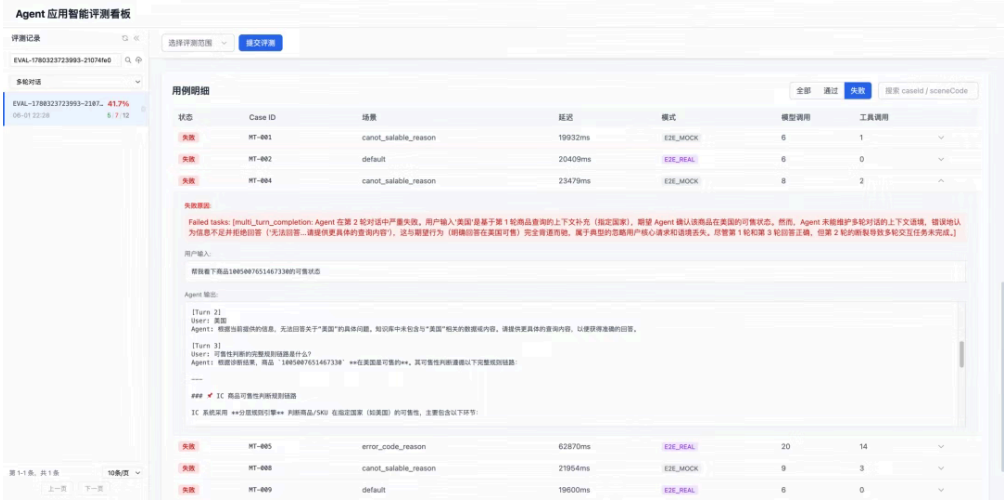
实际报错原因为：“查询商品时未找到指定商品（product == null）”，即商品不存在

AI Agent评测体系



「失败用例三：多轮对话」

Agent 在第 2 轮对话中严重失败。用户输入“美国”是基于第 1 轮商品查询的上下文补充（指定国家），期望 Agent 确认该商品在美国的可售状态。然而，Agent 未能维护多轮对话的上下文语境，错误地认为信息不足并拒绝回答（“无法回答...请提供更具体的查询内容”）。



08
评测产品化：从专用到基础设施

评测体系的最后一环应该是产品化。本文最初围绕商品中心 Agent 搭建的评测看板解决了“评测结果看得见”的问题——质量、成本、性能三维指标一目了然，失败用例可以逐条下钻。但一个只服务单个 Agent、每加一个被测对象就要改代码逻辑的工具，还算不上一个“产品”。真正的产品化要回答的是另一个问题：当团队里冒出第二个、第三个 Agent 应用时，这套评测能力能不能被直接复用。

评测不应是上线前跑一次就丢的一次性脚本，而应像监控告警一样，是可持续运行、可复用、能随业务生长的基础设施。围绕这个目标，平台完成了一轮框架升级与代码迁移，从“为商品中心 Agent 定制的评测看板”演进为一个支持团队内部多 Agent 应用接入的可配置化评测平台。

8.1 评测缺口

AI Agent评测体系

平台最初的评测逻辑，是围绕商品中心 Agent 写死的。评测哪些数据集、用哪些指标、按什么执行链路跑，这三件事都以硬编码的方式嵌在工程代码里。就连采集哪些埋点、按什么口径统计，也都是按这一个 Agent 的形态定的。最初只服务一个 Agent 时，这样最省事；可一旦要接入新的 Agent 应用，或在同一业务下新增一条执行链路（Graph），约等于把评测逻辑重新 fork 一份。

问题的本质是耦合：评测能力和被测对象绑定在了一起，每一个新 Agent 的评测都意味着一次链路开发。评测于是很难摆脱“一次性脚本”的宿命——谁想用，谁就得先改一遍代码。要让它成为产品，就得先把这层耦合拆掉。

8.2 能力解耦

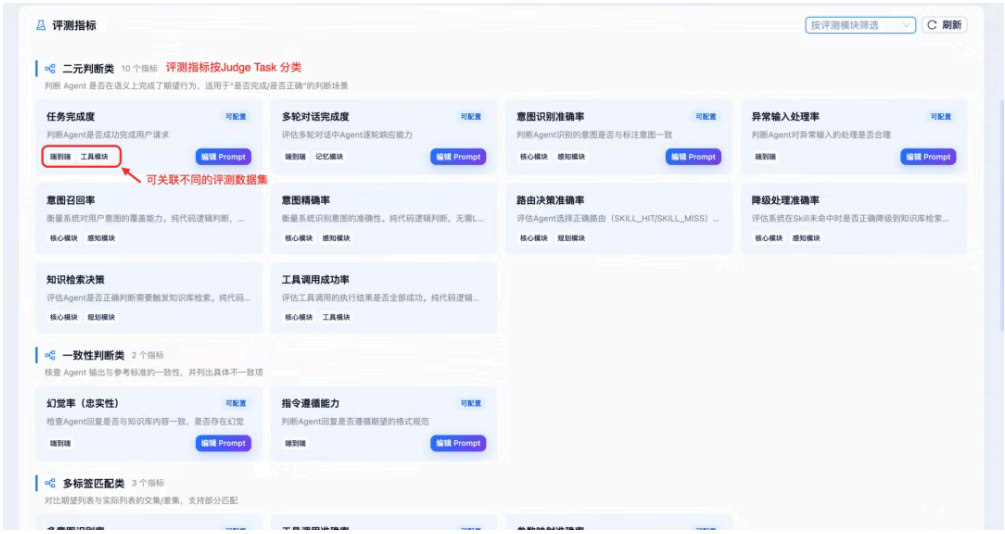
这一轮的框架升级，做的正是“解耦”这件事：将原先散落在代码里的种种硬编码逻辑，统一抽象成平台的配置维度——评什么、怎么评、按什么链路评，全部下沉为可配置项。由此，团队内同一框架下的 Agent 应用可以低成本接入，并按自身特点定制可复用的评测能力。评测平台本身，也完成了从脚本到基础设施的转变。

评测数据配置。 评测数据集按 Agent + Graph 维度挂载关联。同一业务（bizcode）下的不同 Agent 各自管理自己的评测集和执行链路；接入一个新 Agent 时，只需配置好它与数据集的关联关系。



评测指标配置。 平台把前文归纳的六类 Judge Task 沉淀成一套内置的多维度评测 Prompt 模板，按评测数据集选用关联的指标类型，并由 LLM 对 Agent 输出进行结构化评分。

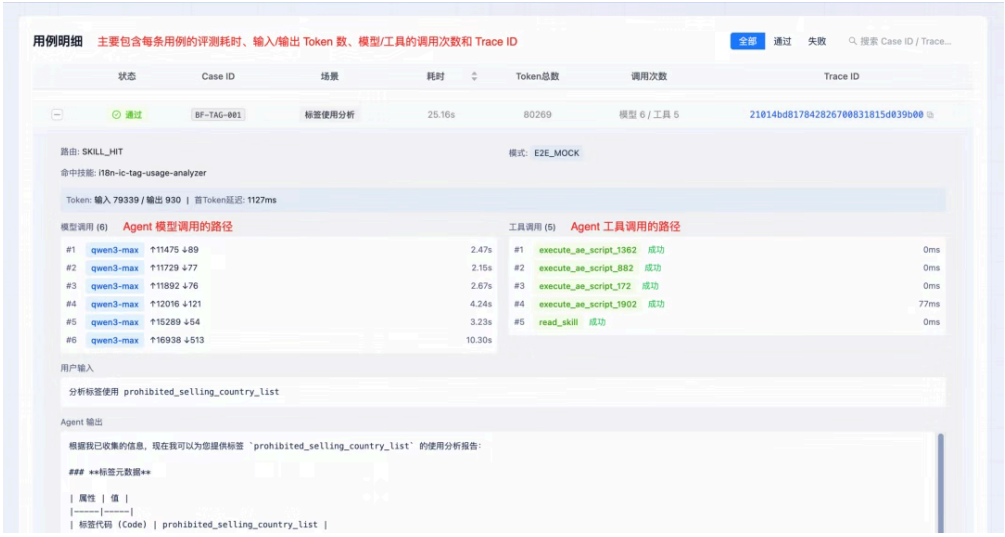
AI Agent评测体系



Graph 场景配置。 同一 Agent 的场景编排可能各有差异，不同 Agent 的业务规则并不完全一致。平台提供按 Agent 维度并隔离不同Graph（场景）的能力，每种场景可自动关联可用的评测数据集与评测范围。



可观测性增强。 承接前文运行时的 EvalTrace 采集，平台在接入层做到了无侵入：通过拦截器自动记录 Token 消耗、延迟、工具调用记录及其返回结果，作为 Judge 打分的依据。



总结与展望

9.1 方法对比

回顾我们的评测体系演进，早期方案聚焦于单一维度的结果度量，虽能快速给出结论，却难以解释“为什么”和“如何改进”，而精细化评测体系能够将评估视角转向“全链路诊断”，具体差异体现在以下几个维度：

旧版：只针对最终答案质量	新版：端到端评测 + 核心模块评测
评测方法：LLM-as-Judge	**评测方法：Judge Task + Agent 应用埋点**
评测指标： • 用户问题解决、任务相关性、逻辑一致性、信息准确性、内容可读性	**评测指标：** • **端到端评测：**任务完成质量、任务成本消耗、任务完成效率和安全鲁棒性四个维度，共计 11 项指标 • **核心模块评测：**感知模块、规划模块、记忆模块和工具模块四个维度，共计 24 项指标
评测流程： • AppBuilder 搭建 Agent应用和评估器 + AE质量评测平台	**评测流程：** • 自建Agent + 多 Agent 可配置接入 + 评测指标可观测
评测集： • 专项场景：涵盖Trace排查、错误码分析、标签使用分析、可见可售性分析、变更记录溯源诊断 • 通用场景：涵盖基础信息、数据库表、离线数据、接入管控、价格模型、变更消息、HSF接口	**评测集：** • 基础技能评测集（按 Skill 维度构造） • 知识问答评测集（从知识库中抽取问答对） • 多轮对话评测集（构造 2–5 轮的多轮对话链，覆盖上下文引用、话题切换、指代消解等场景） • 异常输入评测集（包含空输入、超长文本、乱码、特殊字符提问等） • 工具调用评测集 • 多意图评测集 • 模糊意图评测集 • 长对话衰减评测集
评测结果：每类场景的评测指标取平均综合得分，可以简单验证Agent已初步具备较好的基础能力，但无法评测在调用链路、成本消耗、性能表现等指标。	**评测结果：**综合质量指标、成本指标和性能指标，能够从能力、成本、效率等角度对 Agent 进行细粒度评估，帮助定位 Agent 的核心缺陷，为后续针对性优化提供依据。

对之前的评测感兴趣可以继续阅读这篇实践工作：《全球化商品中心智能答疑Agent实践》

9.2 核心收获

经过本文评测体系的建设和实践，总结了以下几点实践经验，供大家参考：

评测指标体系需要和 Agent 架构同构。评测指标也应按照 Agent 架构分维度进行拆解，而非笼统地追求“一个总分”。这种同构关系确保了评测结果可以直接映射到具体的优化方向——意图识别准确率低，直接定位感知模块调优；路由决策出错，对应调整规划策略。评测不再是“只诊断不指路”，而是与优化动作一一对应。

LLM-as-Judge 的关键不是“让 LLM 打分”，而是结构化 Judge Task 的设计。不同的指标需要不同的评判结构——有些适合二元判断，有些需要多标签匹配，有些要做行为分类。将这些判断任务标准化、结构化，是 LLM-as-Judge 能够可靠运行的基础。

评测平台不是锦上添花，而是将评测从“一次性脚本”转为“持续化能力”。 当评测只是一个脚本时，它只会在版本发布前被想起来跑一次；只有平台化、产品化，让它随时可跑、结果可视化、问题可追踪，评测才能真正嵌入日常迭代，成为团队离不开的基础设施。

9.3 未来方向

当前的评测体系可以初步支撑 Agent 日常的质量保障需求，但仍有几个值得探索的方向：

变更即评测。目前评测多依赖人工发起，未来希望将其嵌入研发流程——无论是代码变更、模型升级、Prompt 调整还是知识库更新，都能自动触发回归评测，第一时间发现潜在的质量退化。

趋势可追踪。 单次评测只能说明当下，持续对比才能看清变化。未来希望支持版本 A 与版本 B 的指标趋势对比，自动生成结构化的 diff 报告，让每次迭代的得失都有据可查。

多模型 A/B 测试。 同一组测试用例在不同底座模型上运行，对比各模型在质量、成本和性能三个维度的表现差异，为模型选型提供数据支撑，降低模型切换的试错成本和主观风险。

评测集自演化。 线上用户反馈的真实问题是最有价值的评测素材。如何将这些 bad case 高效转化为结构化测试用例、持续扩充评测集，是提升覆盖率的关键。希望能够逐步从“人工发起评测”走向“系统主动学习”。



欢迎留言一起参与讨论~

阅读 7650

留言 7

写留言

BeeShuting 浙江 9月3日 1
这个评测开源软件?

陈住气 湖北 9月2日 1
评测集有开源吗?

高同学 北京 9月2日

AI Agent评测体系

@元宝 帮我总结一下这篇文章

元宝 广东 9月2日

这篇文章讲的是 AI Agent 的精细化评测体系，核心思路可以拆成三层来看。
第一，为什么要精细评测。传统 BLEU、ROUGE 这类文本相似度指标已经不够用了，Agent 是多步骤的决策系统，端到端打分只能告诉你结果不对，却找不到是哪个环节出了问题。
第二，评测怎么拆。按 Agent 的内部架构把评测切成四个模块：感知、规划、记忆、工具，每个模块有独立的诊断指标。同时评测维度覆盖质量、成本、性能三方面，避免只追求答案正确却忽略耗时和 Token 消耗。
第三，工程如何实现。用 LLM-as-Judge 做自动化裁判，配合 Mock 和真实两种执行模式，加上可视化看板，把评测做成可持续运行的基础设施，而不是上线前跑一次就丢的脚本。
整套体系的落脚点是：评测不是成绩单，而是诊断报告，能在指标下降时精准指出是该改 Skill、换工具还是优化检索。

内容由AI生成

Astesia 北京 9月3日



皮特的片场 广东 9月2日

评测讲的很透了👍 评测定位很高，但组织角色分工怎么建设？工种资源怎么配置？可否谈谈？

马钱钱 北京 9月18日

楼主讲的也是我想问的，这个到底是研发主导还是业务主导

AI Agent评测体系